Université Grenoble Alpes
L3 Informatique

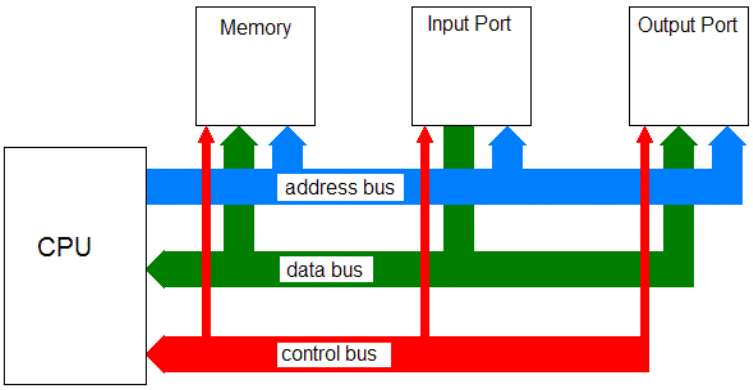
Année 2026-2027

Architectures logicielles et matérielles
Semaine 2 TD (1)

Nous considérons ici les communications d'un microprocesseur avec la **mémoire** et les **périphériques**, et la *notion* ***d'adressage***.
Le processeur devra faire des lectures/écritures en mémoire, pour lire les *instructions* du programme en cours d'exécution, et pour lire/écrire les *données* manipulées par ce programme. Il pourra également réaliser des entrées/sorties avec des périphériques.

Questions à traiter en début de séance. (remise à 8h15)

Question 1.
Dans un microcontrôleur, les communications avec la mémoire (et les périphériques) utilisent un bus de données, un bus d'adresses, et un bus de contrôle.
Vue schématique très simplifiée :



http://microprocessorsystem-graxia.blogspot.com/2010/07/assignment-in-microprocesor-system.html

1.1 Rappeler les signaux de contrôle que nous avons évoqués, et le rôle de chacun d'eux.

1.2 Lors d'une opération d'**écriture** d'une donnée D à une adresse A en mémoire :
- le CPU place-t-il une valeur sur le bus d'adresses ? si oui, laquelle ?
- le CPU place-t-il une valeur sur le bus de données ? si oui, laquelle ?
- le CPU modifie-t-il des signaux du bus de contrôle ? si oui, lesquels, comment, et pourquoi ?

1.3 Lors d'une opération de **lecture** à une adresse A en mémoire :
- le CPU place-t-il une valeur sur le bus d'adresses ? si oui, laquelle ?
- le CPU place-t-il une valeur sur le bus de données ? si oui, laquelle ?
- le CPU modifie-t-il des signaux du bus de contrôle ? si oui, lesquels, comment, et pourquoi ?

Question 2.
2.1 Quelle est la signification de la macro suivante (autrement dit, quel est le type de la constante et que représente-t-elle) ?

```
#define DATA_ADR (volatile uint8_t *)0x400000
```

Où se trouve *l'opérateur de cast* (transtypage) dans cette expression et à quoi sert-il ?

Quel est le rôle du mot clé *volatile* ? Pourquoi sera-t-il indispensable dans un contexte d'accès aux registres de périphériques ?

2.2 Définir, en utilisant la macro `DATA_ADR`, une macro `DATA` qui représentera le contenu de ce même registre.

Exercice 1.

Par la suite, nous allons devoir nous habituer à accéder aux contenus des registres des périphériques via leurs adresses. Il faut donc maîtriser en C les accès aux variables via des pointeurs sur ces variables (nous y reviendrons encore en TP).

Expliquer de quels types sont les variables utilisées dans le code C ci-dessous, et comment elles sont initialisées, modifiées (ainsi que les valeurs pointées le cas échéant - faites bien la différence !) lors de l'exécution de cette portion de code :

```
int *q;
int x=5,y;
int *p = &x;
*p = 3;
p = &y;
*p += 1;
```

Exercice 2.

Dans cet exercice, nous illustrons le fait qu'avec un adressage vers les périphériques fait par *memory mapping*, nous pourrons **accéder de la même façon** (avec les mêmes instructions assembleur) **à la mémoire et aux périphériques**.
Il suffira en C d'accéder aux contenus des registres des périphériques via leurs adresses.

Q1. Considérons le programme jouet ci-dessous (il n'a pas de sens particulier, et les adresses ne sont pas réalistes) :

```
#include <stdint.h>

#define PERIPH1DATA (volatile uint32_t *)0x5600
#define PERIPH1CTRL (volatile uint32_t *)0x5604
#define PERIPH1STAT (volatile uint32_t *)0x5608

#define K 5

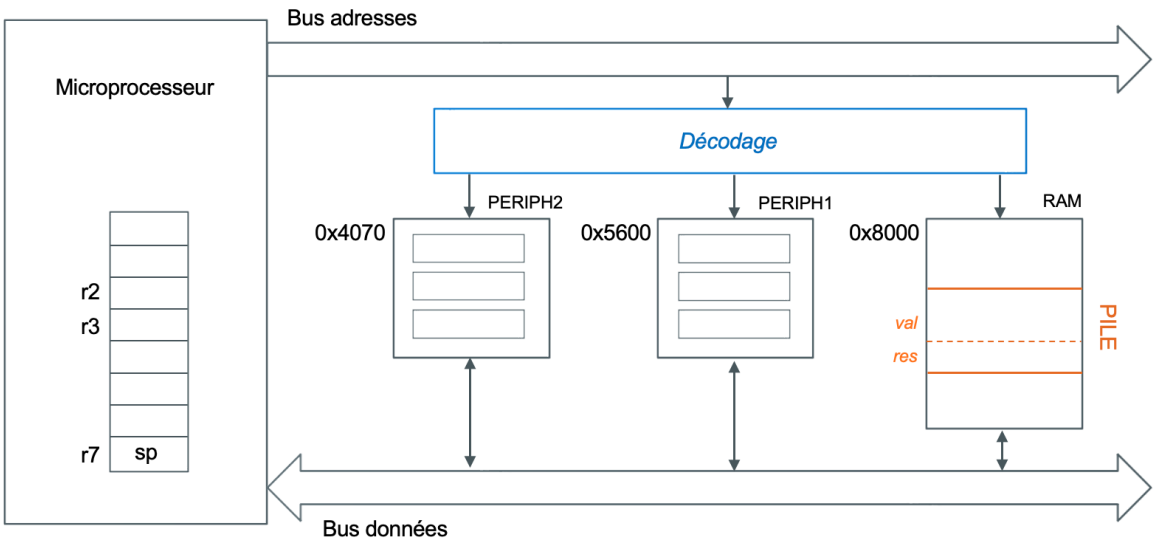
#define PERIPH2DATA (volatile uint32_t *)0x4070

int main()
{
  uint32_t val, res;
  val = *PERIPH1DATA;
  res = val * 6;
  if (*PERIPH1STAT & (1 << K)) {
    *PERIPH2DATA = res;
  }
  else *PERIPH2DATA = 0;
  return 0;
}
```

1.1 Que représentent les **macros** de ce programme ?
Quel est le type des **variables** *val* et *res* ? Et dans quelle zone de mémoire se trouveront ces variables ?
Faudrait-il ajouter le mot-clé "volatile" à la déclaration de ces variables, et pourquoi ?

1.2 **Décrire** soigneusement ce que doit réaliser ce programme, en détaillant sur chaque instruction.

Q2. Nous illustrons ci-dessous le fait qu'avec un adressage vers les périphériques par *memory mapping*, le même espace d'adressage est utilisé pour la mémoire et les périphériques et les accès sont uniformes : toutes les adresses des accès transitent par le bus d'adresses, des plages d'adresses spécifiques correspondent aux périphériques, et l'architecture matérielle assure le décodage pour que l'accès se fasse au bon endroit.
Nous expliciterons cette notion de décodage, et son implémentation matérielle, *dans la deuxième partie de l'UE.*



L'objectif de cette question est d'analyser le lien entre les **instructions load/store** exécutées par le programme et les **transferts** matériellement réalisés, sur ce schéma.

Voyons ci-dessous la correspondance entre notre programme en C et sa compilation en assembleur ARM (sans optimisation) :

```c
#include <stdint.h>

#define PERIPH1DATA (volatile uint32_t *)0x5600
#define PERIPH1CTRL (volatile uint32_t *)0x5604
#define PERIPH1STAT (volatile uint32_t *)0x5608

#define K 5

#define PERIPH2DATA (volatile uint32_t *)0x4070

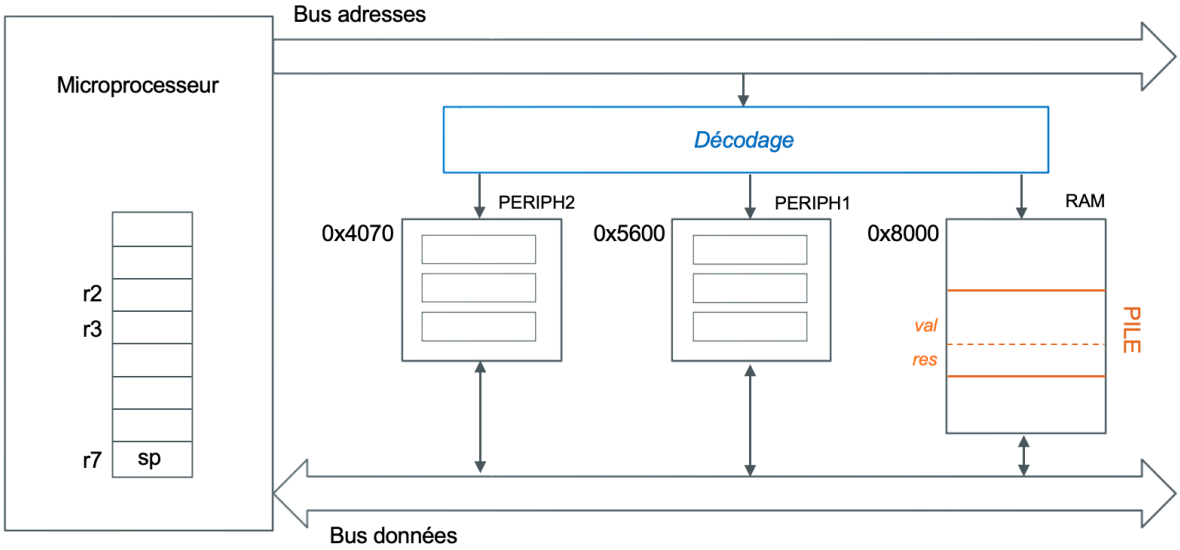
int main(){
  uint32_t val, res;
  val = *PERIPH1DATA;
  res = val * 6;
  if (*PERIPH1STAT & (1 << K)) {
    *PERIPH2DATA = res;
  }
  else *PERIPH2DATA = 0;
  return 0;
}
```

```
main:
        push    {r7}
        sub     sp, sp, #12
        add     r7, sp, #0
        mov     r3, #22016
    1   ldr     r3, [r3]
    2   str     r3, [r7, #4]
    3   ldr     r3, [r7, #4]
        movs    r2, #6
        mul     r3, r2, r3
    4   str     r3, [r7]
        movw    r3, #22024
    5   ldr     r3, [r3]
        and     r3, r3, #32
        cmp     r3, #0
        beq     .L2
        movw    r2, #16496
    6   ldr     r3, [r7]
    7   str     r3, [r2]
        b       .L3
.L2:
        movw    r3, #16496
        movs    r2, #0
    8   str     r2, [r3]
.L3:
        movs    r3, #0
        mov     r0, r3
        adds    r7, r7, #12
        mov     sp, r7
        pop     {r7}
        bx      lr
```

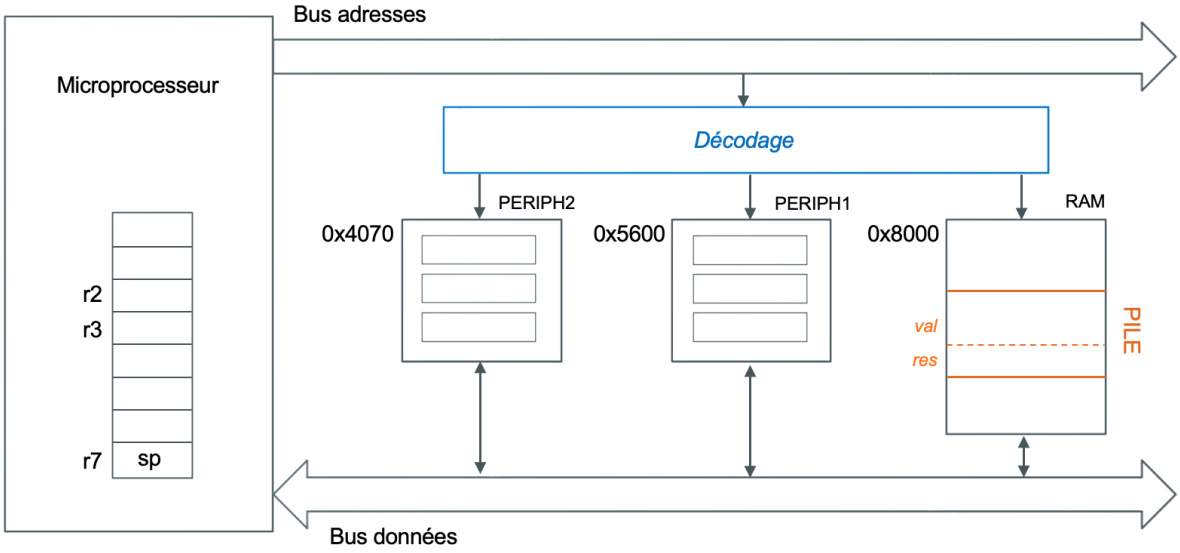
Reprendre votre explication de la question 1 sur **ce que réalise chaque instruction** C de ce programme, dans sa version assembleur ARM, et **représenter** en particulier **sur les schémas**

ci-dessous les transferts réalisés par les instructions numérotées de 1 à 8 (représenter par des flèches l'acheminement des adresses et des données, et écrire les modif. de valeurs).

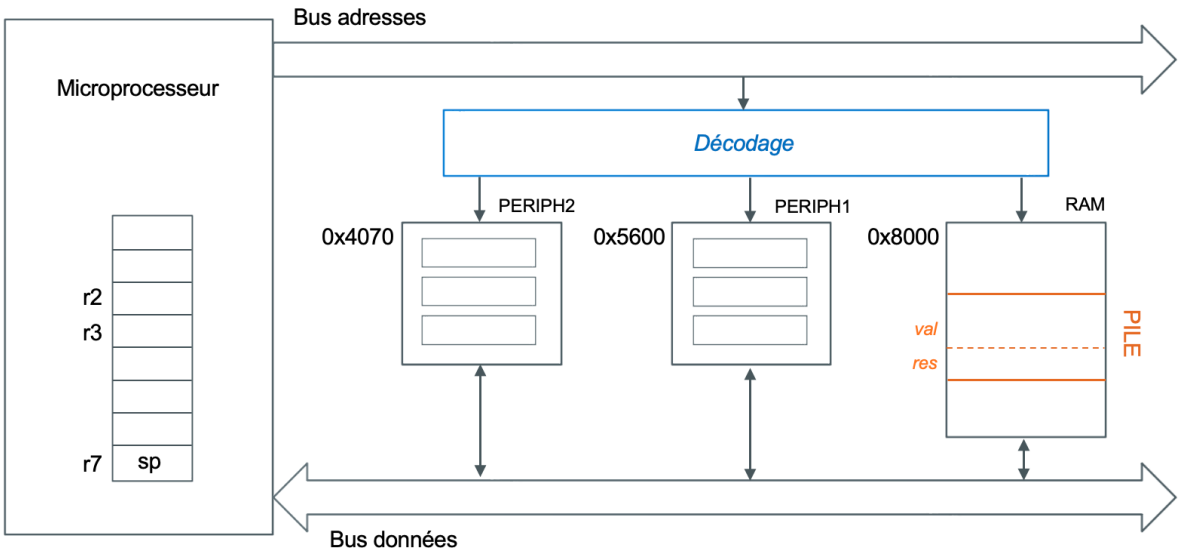
Sur instruction 1



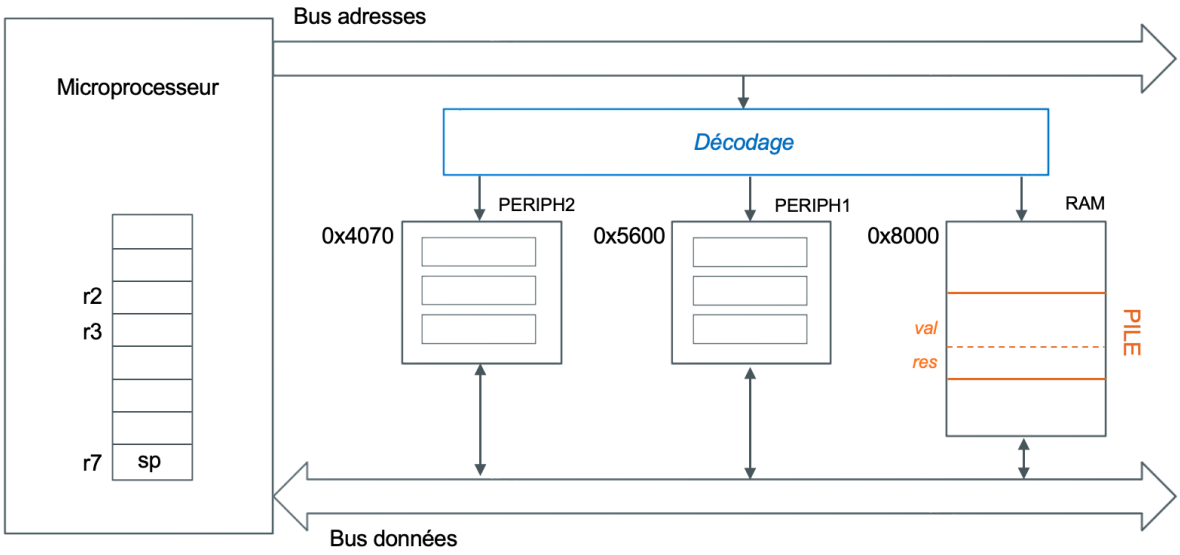
Sur instruction 2



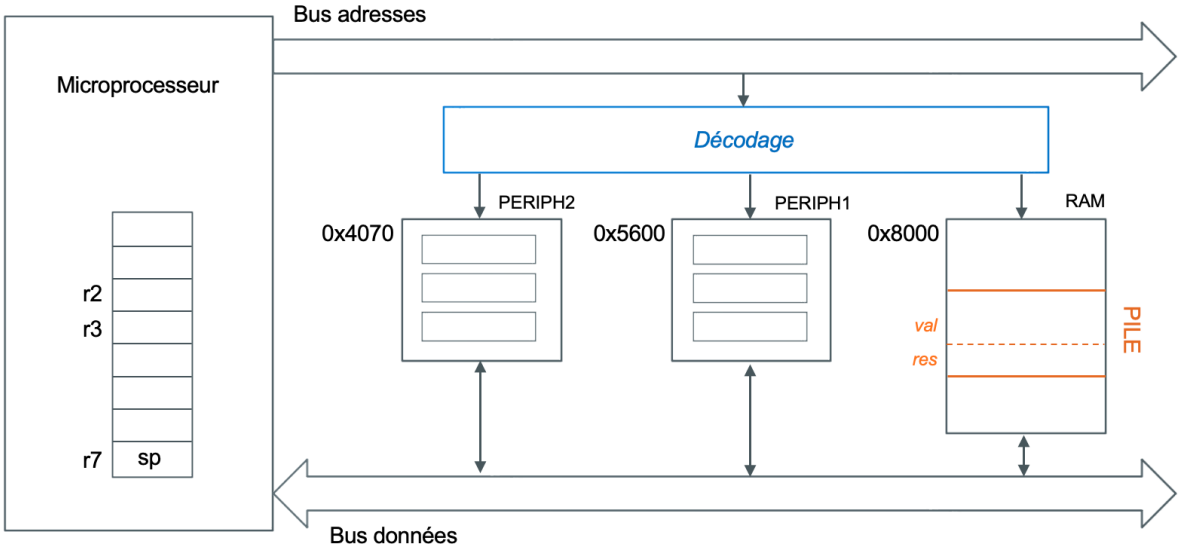
Sur instruction 3



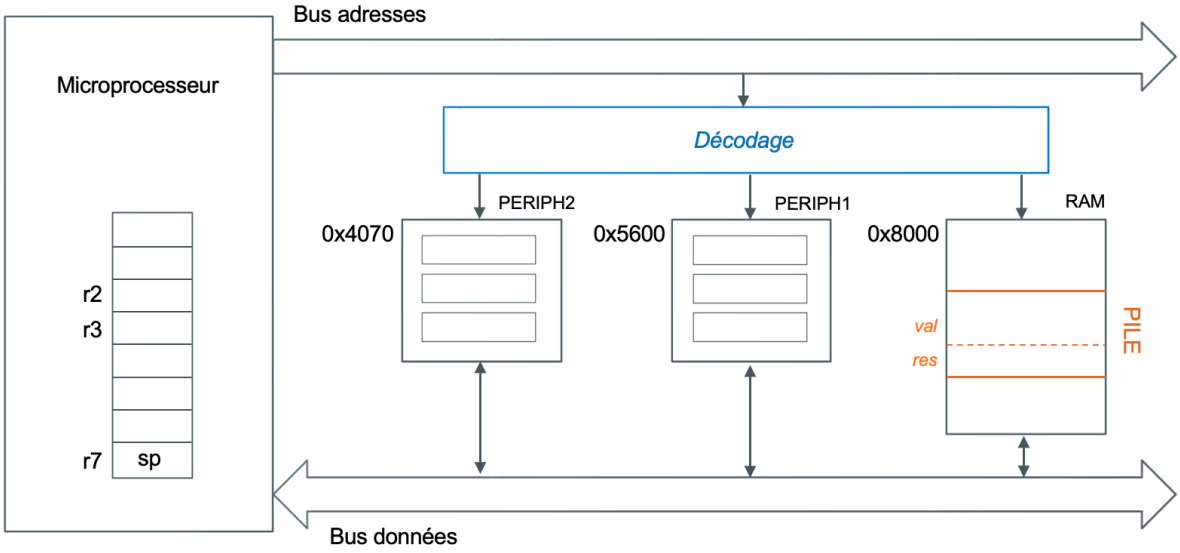
Sur instruction 4



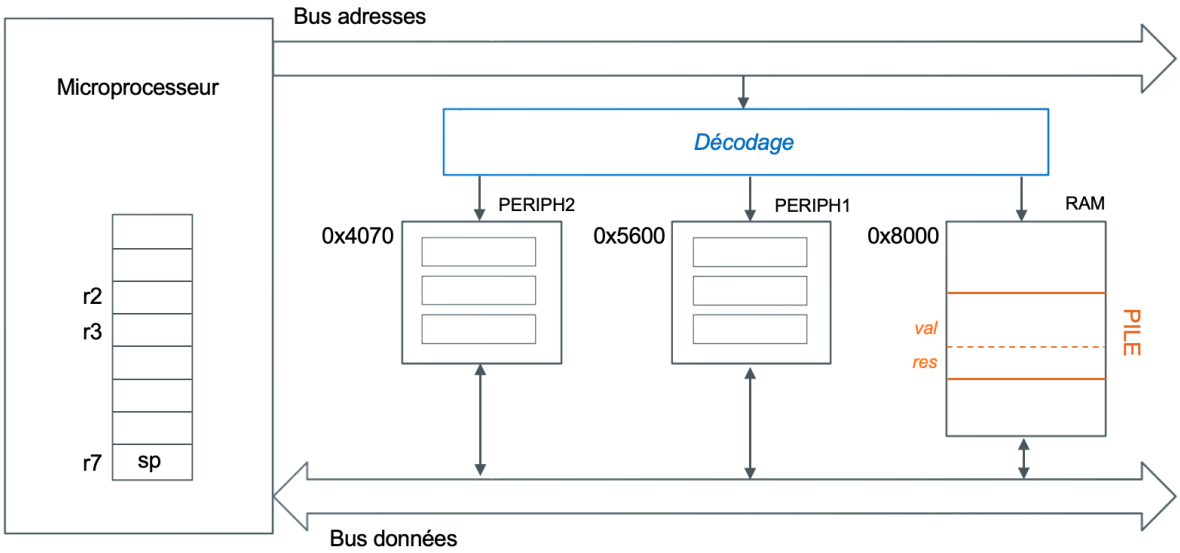
Sur instruction 5



Sur instruction 6



Sur instruction 7



Sur instruction 8

